



CortexAI — Cross-File Parameter Workflow Bible

This document traces every critical parameter that travels across multiple files in CortexAI. For each parameter, you will find: **what it is, where it is born, every file it passes through, and where it finally dies.**

Table of Contents

1. [userid](#)
 2. [session \(cookie + Redis key\)](#)
 3. [token \(Firebase ID Token\)](#)
 4. [x-user-id \(HTTP Header\)](#)
 5. [userData \(Frontend Redux State\)](#)
 6. [conversationId](#)
 7. [prompt](#)
 8. [aiResponse](#)
 9. [agent \(Router Classification Label\)](#)
 10. [messages](#)
 11. [selectedConversation](#)
 12. [Full System Flow Diagram](#)
-

1. `userid`

What it is: The MongoDB `_id` of the authenticated user. This is the single most important parameter in the entire project — it links *everything* (conversations, messages, sessions) to a specific user.

Type: String (MongoDB ObjectId, e.g. `"6832f5a1e4b0c7a2d3f1e9b4"`)

Complete Journey (7 files)

BIRTH

[File 1] services/auth/models/user.model.js
MongoDB auto-generates `_id` when a User document is created.
Schema: { firebaseUid, name, email, avatar }
The `_id` field is implicit – MongoDB adds it automatically.

[File 2] services/auth/controller/auth.controller.js (Lines 12–24)
`user = await User.findOne({ firebaseUid: decoded.uid })`
OR
`user = await User.create({ firebaseUid, name, email, avatar })`
→ `user._id` now holds the MongoDB ObjectId

STORED IN REDIS

[File 3] services/auth/controller/auth.controller.js (Lines 26–32)
`await redis.set(`session-${sessionId}`, JSON.stringify({
 userid: user._id, ← RENAMED from _id to userid HERE
 name: user.name,
 email: user.email,
 avatar: user.avatar
})), "EX", 604800)`

⚠ KEY INSIGHT: The field name changes from `user._id` → `userid`
This renaming happens ONLY here and affects everything downstream.

EXTRACTED FROM REDIS

[File 4] shared/redis/redis.js
The `ioredis` client connects to Redis and provides `.get()` / `.set()`

[File 5] gateway/middlewares/auth.middleware.js (Line 16)
`req.user = JSON.parse(session)`
→ `req.user = { userid: "6832f5a1...", name, email, avatar }`
The field is now accessible as `req.user.userid`

INJECTED INTO HEADER

[File 6] gateway/utils/proxywithheader.js (Line 11)

```

|   proxyReqOpts.headers['x-user-id'] = srcReq.user.userid
|   → userid value is now traveling as an HTTP header
|
CONSUMED BY DOWNSTREAM _____
|
|   [File 7] services/CHAT/controller/chat.controller.js (Lines 7, 18)
|   const userId = req.headers["x-user-id"]
|   → Used to create conversations: Conversation.create({ userId })
|   → Used to query conversations: Conversation.find({ userId })
|
DEATH _____
    Stored permanently in the Conversation document in MongoDB.
    Links every conversation to its owner forever.

```

Name Changes Along the Way

File	Variable Name	Why the name changes
user.model.js	user._id	MongoDB's default field name
auth.controller.js	userid	Renamed during Redis JSON serialization
auth.middleware.js	req.user.userid	Parsed from Redis JSON
proxywithheader.js	srcReq.user.userid → header x-user-id	Injected as HTTP header
chat.controller.js	userId (camelCase)	Read from header, renamed again

2. session (Cookie + Redis Key)

What it is: A UUID string that acts as a session identifier. It exists in two places simultaneously — as a browser cookie and as a Redis key. Together they form the authentication bridge between the browser and the Gateway.

Type: String (UUID v4, e.g. "a1b2c3d4-e5f6-7890-abcd-ef1234567890")

Complete Journey (5 files)

BIRTH

[File 1] services/auth/controller/auth.controller.js (Line 25)

```
const sessionId = crypto.randomUUID()
```

→ A brand new UUID is generated using Node's crypto module

STORED IN TWO PLACES SIMULTANEOUSLY

[File 1 continued] auth.controller.js (Lines 26–32)

Place 1: REDIS

```
redis.set(`session-${sessionId}`, JSON.stringify({
  userid, name, email, avatar
}), "EX", 604800)
```

Key format: "session-a1b2c3d4-e5f6-..."

Value: JSON string of user data

TTL: 604800 seconds (7 days)

Place 2: BROWSER COOKIE

```
res.cookie("session", sessionId, {
  httpOnly: true,    // JS can't read it
  secure: false,     // HTTP allowed (dev mode)
  sameSite: "strict", // No cross-site sending
  maxAge: 604800000  // 7 days in milliseconds
})
```

TRAVELS WITH EVERY REQUEST

[Browser] Automatically attaches cookie header to every request:

Cookie: session=a1b2c3d4-e5f6-...

[File 2] gateway/index.js (Line 21)

```
app.use(cookieParser())
```

→ cookie-parser middleware parses the Cookie header

→ Makes it accessible as req.cookies.session

VALIDATED

```
[File 3] gateway/middlewares/auth.middleware.js (Lines 5-16)
  const sessionId = req.cookies?.session    // Read from cookie
  const session = await redis.get(`session-${sessionId}`) // Lookup in Redis
  req.user = JSON.parse(session)           // Parse and attach

[File 4] shared/redis/redis.js
  The ioredis client performs the actual GET operation against Redis
```

DEATH

```
[File 5] services/auth/controller/auth.controller.js (Lines 56-60)
  On logout:
  redis.del(`session-${sessionId}`) // Delete from Redis
  res.clearCookie("session")       // Delete from browser

OR naturally expires:
  Redis: After 604800 seconds (7 days)
  Cookie: After maxAge expires
```

Why Two Places?

Storage	Purpose
Browser Cookie	The browser needs <i>something</i> to send with every request to prove identity. The cookie holds just the UUID — no sensitive data.
Redis	The actual user data (userid, name, email, avatar) is stored server-side in Redis. The UUID is just a lookup key. Even if someone steals the cookie, they can't see user data from it.

3. token (Firebase ID Token)

What it is: A JWT (JSON Web Token) issued by Firebase after a successful Google login popup. It proves that the user authenticated with Google.

Type: String (JWT, very long — ~1200 characters)

Complete Journey (5 files)

BIRTH

[File 1] frontend/vite-project/utils/firebase.js
 Firebase SDK is initialized with project config.
 auth = getAuth(app)
 googleProvider = new GoogleAuthProvider()
 → These are exported and used by the login flow

[File 2] frontend/vite-project/pages/home.jsx (Lines 29–38)
 const data = await signInWithPopup(auth, googleProvider)
 → Google login popup opens, user signs in
 const token = await data.user.getIdToken()
 → Firebase returns a JWT proving the user's identity
 await handlelogin(token)

SENT TO BACKEND

[File 2 continued] home.jsx (Lines 19–27)
 const { data } = await api.post('/auth/login', { token })
 → Token sent as JSON body to Gateway → proxied to Auth Service

[File 3] gateway/index.js (Line 22)
 app.use("/auth", proxy(process.env.AUTH_SERVICE))
 → /auth route is NOT protected – plain proxy, no auth needed
 → Request forwarded as-is to Auth Service on port 8001

VERIFIED & CONSUMED

[File 4] services/auth/controller/auth.controller.js (Lines 10–11)
 const { token } = req.body
 const decoded = await getAuth(app).verifyIdToken(token)
 → Firebase Admin SDK verifies the JWT signature
 → decoded = { uid, name, email, picture, ... }

[File 5] services/auth/config/firebase.js
 Firebase Admin SDK initialized with serviceAccountKey.json
 → Provides the getAuth() instance used to verify tokens

DEATH

The token is never stored. After verification, only the decoded fields (uid, name, email, picture) are used. The JWT itself is discarded – it served its one-time purpose.

4. `x-user-id` (HTTP Header)

What it is: A custom HTTP header injected by the Gateway into proxied requests. It carries the authenticated user's MongoDB `_id` to downstream services.

Type: String (HTTP header value)

Complete Journey (5 files)

BIRTH

[File 1] gateway/middlewares/auth.middleware.js (Line 16)

req.user = JSON.parse(session)

→ req.user.userid is now available on the request object

[File 2] gateway/utils/proxywithheader.js (Line 11)

proxyReqOpts.headers['x-user-id'] = srcReq.user.userid

→ HEADER IS BORN HERE – injected into the outgoing proxy request

CONSUMED

[File 3] services/CHAT/controller/chat.controller.js

Used in 2 functions:

createConversation (Line 7):

const userId = req.headers["x-user-id"]

Conversation.create({ userId })

→ Links new conversation to the user

getConversations (Line 18):

const userId = req.headers["x-user-id"]

Conversation.find({ userId }).sort({ updatedAt: -1 })

→ Fetches only THIS user's conversations

[File 4] services/CHAT/model/conversation.model.js

userId field in schema: { type: String }

→ Stored permanently in MongoDB conversation documents

[File 5] services/agent/controller/agent.controller.js

Header arrives but is not currently read from headers.

Agent reads prompt and conversationId from req.body instead.

DEATH

HTTP headers exist only for the duration of a single request.

Once the response is sent back, the header is gone.

But its VALUE (the userId) lives on inside MongoDB documents.

5. `userData` (Frontend Redux State)

What it is: The user's profile object stored in the Redux store. Every component that needs to know "who is logged in" reads this value.

Type: Object `{ userid, name, email, avatar }` or `null` (logged out)

Complete Journey (7 files)

BIRTH

```
[File 1] frontend/src/features/getcurrentuser.js
const { data } = await api.get("/me")
return data
→ Calls Gateway GET /me endpoint
→ Gateway's protect middleware validates session
→ Gateway's user.controller.js returns req.user
→ data = { userid, name, email, avatar }
```

DISPATCHED TO REDUX

```
[File 2] frontend/src/App.jsx (Lines 13-19)
const data = await getcurrentuser()
dispatch(setUserdata(data))
→ Fires on every app mount (page refresh)
```

OR after login:

```
[File 3] frontend/pages/home.jsx (Lines 19-23)
const { data } = await api.post('/auth/login', { token })
dispatch(setUserdata(data))
→ data here = { message: "login successful", user: {...} }
```

STORED IN REDUX

```
[File 4] frontend/src/redux/userslice.js
initialState: { userData: null }
reducers: {
  setUserdata: (state, action) => { state.userData = action.payload }
}
```

```
[File 5] frontend/src/redux/store.js
reducer: { user: userReducer, ... }
→ Accessed as state.user.userData
```

CONSUMED BY COMPONENTS

```
[File 6] frontend/pages/home.jsx (Line 14)
const { userData } = useSelector((state) => state.user)
```


- Controls login modal: `{!userData && <LoginModal/> }`
- If `userData` is null, user sees login screen
- If `userData` exists, user sees the app

[File 7] `frontend/src/components/sidebar.jsx` (Lines 14, 158–192)

- ```
const { userData } = useSelector((state) => state.user)
```
- Displays user avatar: `userData?.avatar`
  - Displays user name: `userData?.name`
  - Displays user email: `userData?.email`
  - Controls profile section vs "Login" button

DEATH

---

[File 7] `frontend/src/components/sidebar.jsx` (Lines 30–33)

- ```
handleLogout:
  await logout()           // Clears session on backend
  dispatch(setUserdata(null)) // Clears from Redux
```
- `userData` becomes null → login modal reappears

Also dies on page refresh (Redux state is in-memory only),
but `App.jsx` re-fetches it from `/me` on mount.

6. conversationId

What it is: The MongoDB `_id` of a Conversation document. It links messages to their parent conversation and travels from the frontend all the way through the Agent service to the Chat service.

Type: String (MongoDB ObjectId)

Complete Journey (8 files)

BIRTH

[File 1] services/CHAT/controller/chat.controller.js (Lines 5–13)

```
const conversation = await Conversation.create({ userId })
```

→ MongoDB generates conversation._id automatically

→ Returned to frontend as JSON

[File 2] services/CHAT/model/conversation.model.js

```
Schema: { title: "New Chat", userId: String }
```

→ _id is auto-generated by MongoDB

ARRIVES AT FRONTEND

[File 3] frontend/src/features/createconversation.js

```
const { data } = await api.get("/chat/create-conversation")
```

```
return data // data = { _id: "abc123...", title: "New Chat", userId: "..." }
```

[File 4] frontend/src/components/sidebar.jsx (Lines 25–28)

```
const data = await createConversation()
```

```
dispatch(addConversation(data))
```

→ Conversation object (with _id) added to Redux

STORED IN REDUX

[File 5] frontend/src/redux/conversationslice.js

```
addConversation: state.conversations.unshift(action.payload)
```

```
setSelectedConversation: state.selectedConversation = action.payload
```

→ When user clicks a conversation in sidebar:

selectedConversation._id becomes the active conversationId

SENT TO AGENT SERVICE

[File 6] frontend/src/components/chatinput.jsx (Lines 13–14)

```
const payload = {
```

```
  prompt: input.trim(),
```

```
  conversationId: selectedConversation?._id ← READ FROM REDUX
```

```
}
```

```
const data = await sendmessage(payload)
```

```
| [File 7] frontend/src/features/sendmessage.js
|   const { data } = await api.post("/agent/chat", payload)
|   → POST to Gateway → proxied to Agent Service
|
```

USED BY AGENT TO SAVE MESSAGES

```
| [File 8] services/agent/controller/agent.controller.js (Lines 6–20)
|   const { prompt, conversationId } = req.body
```

```
|   // Save user message:
|   await axios.post(`${CHAT_SERVICE}/save-message`, {
|     conversationId,    ← FORWARDED to Chat Service
|     role: "user",
|     content: prompt
|   })
```

```
|   // Run AI, then save AI response:
|   await axios.post(`${CHAT_SERVICE}/save-message`, {
|     conversationId,    ← FORWARDED again
|     role: "assistant",
|     content: aiResponse
|   })
```

```
| [Back to File 1] chat.controller.js (saveMessage, Lines 37–44)
```

```
|   const { conversationId, role, content } = req.body
|   Message.create({ conversationId, content, role })
|   → Stored permanently in Message document
|
```

DEATH

Lives forever in MongoDB – in the Conversation document itself
and in every Message document that references it.

7. prompt

What it is: The user's text input — the question or instruction they type into the chat. This is the primary input to the entire AI pipeline.

Type: String (user text)

Complete Journey (7 files)

BIRTH

```
[File 1] frontend/src/components/chatinput.jsx (Lines 11, 31)
  const [input, setInput] = React.useState('')
  <textarea onChange={(e) => setInput(e.target.value)} value={input} />
  → User types text, stored in local React state as `input`

[File 1 continued] chatinput.jsx (Lines 13-14)
  const payload = { prompt: input.trim(), conversationId: ... }
  → RENAMED from `input` → `prompt` HERE
```

SENT TO BACKEND

```
[File 2] frontend/src/features/sendmessage.js
  const { data } = await api.post("/agent/chat", payload)
  → { prompt: "Explain Redis", conversationId: "abc123" }

[File 3] gateway/index.js → gateway/utils/proxywithheader.js
  → Proxied to Agent Service on port 8003
```

ENTERS LANGGRAPH

```
[File 4] services/agent/controller/agent.controller.js (Lines 6, 14)
  const { prompt, conversationId } = req.body
  const result = await graph.invoke({ prompt, conversationId })
  → prompt enters the LangGraph state machine

[File 5] services/agent/graph/state.js
  agentState = Annotation.Root({
    prompt: Annotation(),      ← DECLARED as graph state field
    aiResponse: Annotation(),
    agent: Annotation(),
    conversationId: Annotation()
  })

[File 6] services/agent/graph/router.js (Line 59)
  User Query: ${state.prompt}
  → prompt is embedded into the router's LLM prompt
  → LLM classifies intent → returns agent name
```

```
|  
| [File 7] services/agent/agents/chat.agent.js (Line 14)  
|   { "role": "human", "content": state.prompt }  
|   → prompt becomes the user message sent to the LLM  
|   → LLM generates aiResponse based on this prompt  
|
```

DEATH

After LLM processing, the prompt is saved to MongoDB as a message with role: "user" (via agent.controller → chat service).
Lives forever in the Message collection.

8. aiResponse

What it is: The AI-generated text response from the LLM. This is the output of the entire AI pipeline and travels backward from the agent to the frontend.

Type: String (LLM-generated text)

Complete Journey (7 files, reverse direction)

BIRTH

```
[File 1] services/agent/agents/chat.agent.js (Lines 6-21)
  const response = await llm.invoke([system_msg, user_msg])
  return { ...state, aiResponse: response.content }
  → BORN HERE – LLM's text output becomes aiResponse
```

```
[File 2] services/agent/config/llmmodel.js
  getModel("chat") returns the Groq LLM instance
  → Currently using "openai/gpt-oss-120b" via Groq API
```

FLows THROUGH LANGGRAPH

```
[File 3] services/agent/graph/state.js
  aiResponse: Annotation()
  → State field updated by the chat agent node
```

```
[File 4] services/agent/graph/graph.js
  workflow: chat node → _end_ node
  const graph = workflow.compile()
  → Graph execution completes, result contains aiResponse
```

EXTRACTED BY CONTROLLER

```
[File 5] services/agent/controller/agent.controller.js (Lines 14-22)
  const result = await graph.invoke({ prompt, conversationId })
  const aiResponse = result.aiResponse

  // Saved to Chat Service:
  await axios.post(`${CHAT_SERVICE}/save-message`, {
    conversationId, role: "assistant", content: aiResponse
  })

  // Sent back to frontend:
  return res.status(200).json({ aiResponse })
```

ARRIVES AT FRONTEND

```
[File 6] frontend/src/features/sendmessage.js
```

```
|   const { data } = await api.post("/agent/chat", payload)
|   return data    // data = { aiResponse: "Redis is an in-memory..." }
|
| [File 7] frontend/src/components/chatinput.jsx (Line 23)
|   const data = await sendMessage(payload)
|   dispatch(addMessage({ role: 'assistant', content: data.aiResponse }))
|   → Added to Redux messages array
|   → MessageList re-renders → MessageBubble displays it
|
```

DEATH

Lives in two places forever:

1. MongoDB Message collection (role: "assistant")
2. Redux state (until page refresh)

9. agent (Router Classification Label)

What it is: A single-word string that the router LLM produces to classify which specialized agent should handle the user's query. It only exists inside the LangGraph state machine.

Type: String — one of: "chat", "search", "coding", "pdf", "ppt", "vision"

Complete Journey (4 files, internal only)

BIRTH

```
[File 1] services/agent/graph/router.js (Lines 3-70)
  const response = await llm.invoke(classificationPrompt)
  return { ...state, agent: response.content.trim().toLowerCase() }
  → LLM returns one word (e.g. "chat")
  → Trimmed, lowercased, stored in state.agent
```

USED FOR CONDITIONAL ROUTING

```
[File 2] services/agent/graph/graph.js (Lines 22-47)
  workflow.addConditionalEdges("router", (state) => {
    switch (state.agent) {
      case "chat": return "chat"
      case "search": return "search"
      case "coding": return "coding"
      // ... etc
    }
  })
  → state.agent determines which node executes next
```

```
[File 3] services/agent/graph/state.js
  agent: Annotation()
  → Declared as a state field in the graph schema
```

```
[File 4] services/agent/config/llmmodel.js
  getModel(agent) – but note: router calls getModel("router")
  which falls to default case → returns groq
  → The agent field is NOT currently used for model selection
```

DEATH

Dies when the graph execution completes. It is part of the LangGraph state but is never sent to the frontend or stored in any database. It is purely an internal routing signal.

10. messages

What it is: An array of message objects displayed in the chat UI. Each message has a `role` (user/assistant) and `content` (text).

Type: Array of

```
{ role: "user"|"assistant", content: String, conversationId, _id, createdAt }
```

Complete Journey (7 files)

BIRTH (from database)

[File 1] services/CHAT/model/message.model.js
Schema: { conversationId (ref), role (enum), content, timestamps }

[File 2] services/CHAT/controller/chat.controller.js (Lines 47-53)
getMessage: Message.find({ conversationId }).sort({ createdAt: -1 })
→ Returns all messages for a conversation, newest first

FETCHED BY FRONTEND

[File 3] frontend/src/features/getmessages.js
const { data } = await api.get(`/chat/get-messages/\${id}`)
return data

[File 4] frontend/src/components/chatarea.jsx (Lines 12-19)
const data = await getmessages(selectedConversation?._id)
dispatch(setMessages(data))
→ Fetched when selectedConversation changes

STORED IN REDUX

[File 5] frontend/src/redux/messageslice.js
initialState: { messages: [] }
setMessages: state.messages = action.payload // Replace all
addMessage: state.messages = [...state.messages, action.payload] // Append one

RENDERED IN UI

[File 6] frontend/src/components/messagelist.jsx (Lines 8, 33-38)
const { messages } = useSelector(state => state.messages)
messages?.map((msg) => <MessageBubble role={msg.role} content={msg.content} />)

[File 7] frontend/src/components/messagebubble.jsx (Lines 6-24)
function MessageBubble({ role, content })
→ role determines styling (user = purple gradient, assistant = dark bg)
→ content displayed inside the bubble

GROWS IN REAL-TIME

```
| [Back to File 4] chatinput.jsx (Lines 20-23)
|   dispatch(addMessage({ role: 'user', content: input.trim() }))
|   // ... await AI response ...
|   dispatch(addMessage({ role: 'assistant', content: data.aiResponse }))
|   → Messages added optimistically (user msg) then after API (AI msg)
```

DEATH

Redux array resets when switching conversations (setMessages replaces the array). But messages live forever in MongoDB.

11. selectedConversation

What it is: The conversation object the user has currently clicked on in the sidebar. It controls what messages are displayed and where new messages are sent.

Type: Object { `_id`, `title`, `userId`, `createdAt`, `updatedAt` } or `null`

Complete Journey (6 files)

BIRTH

[File 1] frontend/src/components/sidebar.jsx (Line 43, 58)
onClick={() => dispatch(setSelectedConversation(conv))}
→ User clicks a conversation item in the sidebar
→ The full conversation object is dispatched

STORED IN REDUX

[File 2] frontend/src/redux/conversationslice.js (Lines 15-17)
setSelectedConversation: (state, action) => {
 state.selectedConversation = action.payload
}

CONSUMED BY 4 COMPONENTS

[File 3] frontend/src/components/chatarea.jsx (Line 10)
const { selectedConversation } = useSelector(state => state.conversation)
→ Triggers useEffect to fetch messages when it changes
→ Passes selectedConversation._id to getmessages()

[File 4] frontend/src/components/navbar.jsx (Line 7)
const { selectedConversation } = useSelector(state => state.conversation)
→ Displays: selectedConversation?.title || "New Chat"
→ Only renders if selectedConversation is not null

[File 5] frontend/src/components/messagelist.jsx (Line 7)
const { selectedConversation } = useSelector(state => state.conversation)
→ Controls empty state: if !selectedConversation → show welcome screen

[File 6] frontend/src/components/chatinput.jsx (Line 8)
const { selectedConversation } = useSelector(state => state.conversation)
→ Used to build payload: conversationId: selectedConversation?._id
→ This is how the frontend knows WHICH conversation to send messages to

DEATH

Replaced when user clicks a different conversation.
Set to null on page refresh (Redux resets).
The conversation itself lives forever in MongoDB.

12. Full System Flow Diagram

This shows how ALL the parameters flow together when a user sends a message:

FRONTEND

User types in ChatInput

```
input = "Explain Redis"
```

```
selectedConversation._id = "conv123"
```

Builds payload:

```
{ prompt: "Explain Redis", conversationId: "conv123" }
```

```
dispatch(addMessage({ role: "user", content: "Explain Redis" })))
```

```
sendMessage(payload) → POST /agent/chat
```

```
Cookie: session=<uuid>
```

|



GATEWAY :8000

```
cookieParser() extracts: req.cookies.session = <uuid>
```

```
protect():
```

```
Redis GET session-<uuid> → { userid, name, email, avatar }
```

```
req.user = { userid: "user456", name: "Dhruv", ... }
```

```
proxyWithHeader():
```

```
Adds header: x-user-id = "user456"
```

```
Forwards to http://localhost:8003/chat
```

|



AGENT SERVICE :8003

```
agent.controller.js:
```

```
{ prompt, conversationId } = req.body
```

```
┌ Save user msg to Chat Service ─┐
```

```
└ POST http://localhost:8002/save-message
```

```
  { conversationId, role: "user", content: prompt }
```

```

graph.invoke({ prompt, conversationId })
  |
  | router node: state.prompt → LLM → state.agent = "chat"
  |
  | chat agent: state.prompt → LLM → state.aiResponse = "..."
  |

aiResponse = result.aiResponse

┌ Save AI msg to Chat Service ─┐
| POST http://localhost:8002/save-message
| { conversationId, role: "assistant", content: aiResponse }
└────────────────────────────────┘

return res.json({ aiResponse })

```



FRONTEND (Redux)

```

data = { aiResponse: "Redis is an in-memory data store..." }
dispatch(addMessage({ role: "assistant", content: data.aiResponse }))

Redux messages array:
[ { role: "user", content: "Explain Redis" },
  { role: "assistant", content: "Redis is an in-memory..." } ]

MessageList re-renders → MessageBubble displays both messages

```

Quick Reference: Parameter × File Matrix

Parameter	auth.controller	auth.middleware	proxywithheader	chat.controller
userid	✔ Created	✔ Parsed	✔ Injected	✔ Read
session	✔ Created	✔ Validated	—	—
token	✔ Verified	—	—	—
x-user-id	—	—	✔ Injected	✔ Read
userData	✔ Source	—	—	—
conversationId	—	—	—	✔ Stored
prompt	—	—	—	—
aiResponse	—	—	—	✔ Saved
agent	—	—	—	—
messages	—	—	—	✔ CRUD
selectedConversation	—	—	—	—